# Building a Weather Prediction Analytics using Snowflake and Airflow
# Project Report

Sang Ah Lee[1], Ananya Yallapragada[2]

[1,2]*Data 226: Data Warehouse, San José State University*
*Address Including Country Name*
[1]sangah.lee@sjsu.edu
[2]ananya.yallapragada@sjsu.edu
[1,2]*One Washington Square*
*San José, California 95192, California, USA*

***Abstract*— This project focuses on data pipeline, the usage of DAG, and ETL to analyze weather data and forecast weather.**

***Keywords*— ETL, data pipeline, database, weather forecast, DAG.**

## I. Problem Statement

Weather forecasting is an important way to take into consideration the diverse situations one can face, whether it is anything from every-day planning, to figuring out when the best day or month to plant crops is. To incorporate the topic of Data Analysis, a project of analyzing the weather data from Open-Meteo API to forecast the daily maximum temperature using time-series models was conducted. The project works to analyze given cities, and focuses on fetching the last 60 days of historical weather data, load it into Snowflake using a targeted metric of maximum temperature (in Celsius), and predict the maximum temperature for the next seven days for that specific city. Docker and Airflow were used as tools to make the analysis of three cities possible— Fremont, San Francisco, and Cupertino.

A database is needed to store the historical data, and enables the usage of complex queries. Additionally, as weather observations are infinitely growing, a cloud-based database and data warehouse service such as Snowflake is used to store data and prevent data loss. As this data increases, to start analyzing data, a data pipeline makes it efficient for handling the data and allows for automation. Instead of manually collecting data, one can ensure that data is collected during a specific time, and ETL, a specific process that happens inside a data pipeline, is able to retrieve the raw data, transform them after cleaning and formatting, and load it to a database.

## II. Solution Requirements

### *A. Functional Requirements*

The following are the functional requirements for the system:

- The system must support multiple locations.
- It must automatically fetch the historical data from the last 60 days from the Open-Meto API.
- The fetched data must be stored in Snowflake.
- The system must forecast the maximum temperature of the next seven days.
- The historical data and forecasted results must be stored in the analytics table.

- It must run automatically using Airflow DAGs.

### *B. Non-Functional Requirements*

The system must execute tasks without manual intervention and should have robust error handling through logs. The extraction of daily data should complete within a few minutes. Moreover, the credentials must be managed securely.

### *C. Operational Requirements*

Airflow must run with both the DAGs enabled. Snowflake warehouse and account must be active and accessible. For this, an active internet connection is necessary to call Open-Meteo API.

### *D. System Limitations*

For this project, forecasting is limited to maximum temperature only, and for the next seven days. The data also depends on the availability of the Open-Meteo API. If the service is down, the system cannot fetch the data. Furthermore, the system runs in batches once daily and not real time. So, it does react immediately to new weather observations.

### *E. User Interaction and Usage*

Users will query the analytics tables to view the historical data and the forecasted maximum temperatures. Data Analysts can visualise the trends by using any BI tools. The system retrains and updates the forecasts each day.

## III. Functional Analysis

### *A. Overall System Diagram and Data Flow*

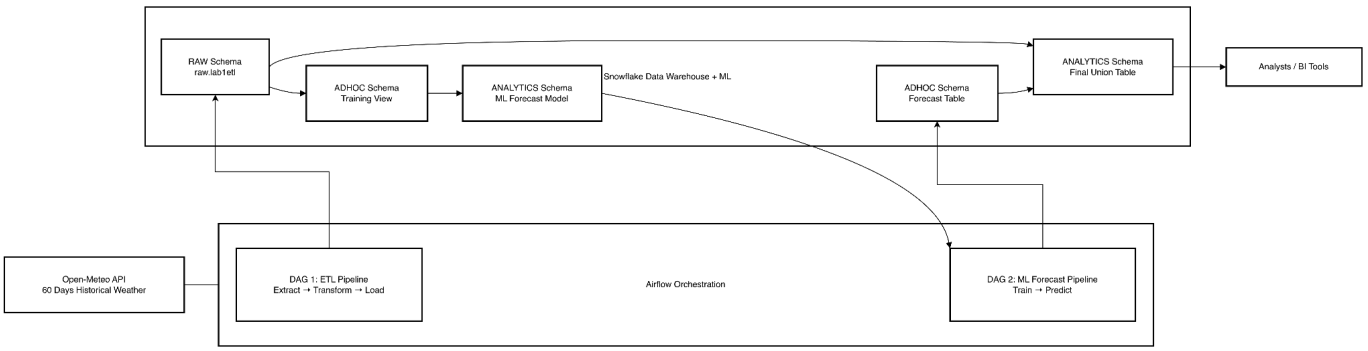


Fig. 1 Overall System Diagram

### *B. Data Ingestion*

This component retrieves historical weather data from Open-Meteo API. It is implemented in *LAB1_ETL* DAG and uses HTTP requests to fetch daily weather data. The inputs are latitude, longitude (MULTI_LOCATIONS variable), API endpoint, 60 days parameter. The API returns a JSON response which contains temperature_2m_max, temperature_2m_min, temperature_2m_mean, precipitation_sum, weather_code, date. The output is then passed to the transform task. To handle errors, API request failure raises RuntimeError. The missing 'daily' field in the response raises ValueError. In Airflow, the logs capture the details of the errors.

*C. ETL/ELT Processing*

The transform function converts the JSON into records. Latitude, longitude, city and date are added. API fields are mapped to the database schema. The current strategy of the system is to insert 60 days of daily data. The primary key is used on latitude, longitude, date and day to prevent duplicate records. This ensures that there is a rolling refresh of daily data. If failure occurs, the system performs 'ROLLBACK'. Try/except blocks are also used to handle errors cleanly.

*D. Data Warehouse (Snowflake)*

Schemas used are raw, adhoc and analytics. Raw stores ingested data, adhoc acts as a training view and forecast table, and analytics is the final output table. Airflow connects using 'snowflake_con' as the connection id.

*E. ML Forecasting Pipeline*

This is implemented in the *lab1_forecast* DAG. The training view includes DS (date), TEMP_MAX (target variable) and CITY (identifies the series). Training uses SNOWFLAKE.ML.FORECAST and the model is stored in 'analytics.lab1_predict_city_weather'. Inference produces the forecast, lower_bound and upper_bound. The lower and upper bounds are the range in which the future value is expected to fall, with a certain probability.

*F. Final Integration Table*

The final table 'analytics.lab1_city_weather_final' is created using historical and forecasted data, and performing a UNION ALL operation on them. The schema aligns the column such that the forecast column of historical data is NULL and the actual column of forecast is NULL. This creates a unified table.

*G. Airflow Orchestration and Scheduling*

The system uses two daily Airflow DAGs. DAG 1 is the ETL pipeline. It extracts the last 60 days of weather data, transforms it and loads it into the Snowflake raw table. DAG 2 is the ML forecasting pipeline which retrains the ML model, generates a seven day forecast and updates the final analytics table after combining the historical and predicted data. The credentials for Snowflake are managed securely through the Airflow connection. The city configurations are stored in the variable MULTI_LOCATIONS on Airflow. The scheduling of the tasks, logging, and monitoring is handled directly by Airflow and the errors (failures) are tracked through the logs.

## IV. Tables Structure, Screenshots, Python Codes, and SQL Queries

*A. Database Table Structures (All Tables)*

*Table 1 : ADHOC.LAB1_CITY_WEATHER_FORECAST*

| Column Name | Data Type | Description | Constraint |
|---|---|---|---|
| SERIES | VARCHAR(100) | City name (time-series identifier) | Not Null |
| TS | TIMESTAMP | Forecasted date/time | Not Null |
| FORECAST | FLOAT | Predicted maximum temperature | - |

| | | | |
|---|---|---|---|
| LOWER_BOUND | FLOAT | Lower bound of 95% prediction interval | - |
| UPPER_BOUND | FLOAT | Upper bound of prediction interval | - |

*Table 2 : ADHOC.LAB1CITY_WEATHER_VIEW*

| Column Name | Data Type | Description | Constraint |
|---|---|---|---|
| DS | DATE | Historical date (time-series timestamp) | Not Null |
| TEMP_MAX | FLOAT | Daily maximum temperature (target variable) | - |
| CITY | VARCHAR(100) | City name (series identifier) | Not Null |

USER_DB_COBRA / ADHOC / LAB1CITY_WEATHER_VIEW

View TRAINING_ROLE 4 minutes ago

View Details Columns Data Preview Data Quality PREVIEW Lineage

COBRA_QUERY_WH Updated just now

| | DS | TEMP_MAX | CITY |
|---|---|---|---|
| 1 | 2026-01-12 | 17.2 | Cupertino |
| 2 | 2026-01-13 | 20.3 | Cupertino |
| 3 | 2026-01-20 | 16.3 | Cupertino |
| 4 | 2026-01-24 | 18.2 | Cupertino |
| 5 | 2026-02-07 | 20.1 | Cupertino |
| 6 | 2026-02-10 | 17.5 | Cupertino |
| 7 | 2026-02-18 | 10.3 | Cupertino |
| 8 | 2026-02-25 | 21.3 | Cupertino |
| 9 | 2026-02-02 | 21.8 | Cupertino |
| 10 | 2026-02-04 | 24.4 | Cupertino |
| 11 | 2026-02-11 | 14.1 | Cupertino |
| 12 | 2026-02-15 | 16.2 | Cupertino |
| 13 | 2026-01-04 | 12.9 | San Francisco |
| 14 | 2026-01-09 | 13.1 | San Francisco |
| 15 | 2026-01-12 | 14.7 | San Francisco |
| 16 | 2026-01-16 | 19.1 | San Francisco |
| 17 | 2026-01-25 | 12.3 | San Francisco |
| 18 | 2026-02-03 | 20.3 | San Francisco |
| 19 | 2026-02-25 | 17.6 | San Francisco |
| 20 | 2026-02-19 | 10.5 | San Francisco |
| 21 | 2026-01-03 | 17 | Fremont |
| 22 | 2026-01-10 | 15.2 | Fremont |
| 23 | 2026-01-24 | 18.7 | Fremont |
| 24 | 2026-01-26 | 18.4 | Fremont |
| 25 | 2026-01-28 | 18.9 | Fremont |
| 26 | 2026-02-03 | 21.3 | Fremont |
| 27 | 2026-02-17 | 9.1 | Fremont |
| 28 | 2026-02-24 | 22 | Fremont |
| 29 | 2026-01-28 | 18.4 | Cupertino |
| 30 | 2026-01-29 | 20.6 | Cupertino |
| 31 | 2026-02-10 | 14 | San Francisco |
| 32 | 2026-01-06 | 16.7 | Fremont |

Fig. 2 ADHOC.LAB1CITY_WEATHER_VIEW Table

*Table 3 : ANALYTICS.LAB1_CITY_WEATHER_FINAL*

| Column Name | Data Type | Description | Constraint |
|---|---|---|---|
| CITY | VARCHAR(100) | City name (time-series identifier) | Not Null |
| DATE | TIMESTAMP_NTZ | Past date/time | Not Null |
| ACTUAL_MAX | FLOAT | Actual maximum temperature | - |
| TEMP_MIN | FLOAT | Actual minimum temperature | - |
| ACTUAL_MEAN | FLOAT | Actual mean temperature | - |
| PRECIPITATION | FLOAT | Actual precipitation amount | - |
| FORECAST | FLOAT | Predicted maximum temperature | - |
| LOWER_BOUND | FLOAT | Lower bound of 95% prediction interval | - |
| UPPER_BOUND | FLOAT | Upper bound of prediction interval | - |

USER_DB_COBRA / ANALYTICS / LAB1_CITY_WEATHER_FINAL

Load Data

Table | TRAINING_ROLE | 3 minutes ago | 201 | 16.5KB

Table Details | Columns | Data Preview | Copy History | Data Quality PREVIEW | Lineage

COBRA_QUERY_WH | 100 of 201 Rows • Updated just now

| | CITY | DATE | ACTUAL_MAX | TEMP_MIN | ACTUAL_MEAN | PRECIPITATION | FORECAST | LOWER_BOUND | UPPER_BOUND |
|---|---|---|---|---|---|---|---|---|---|
| 1 | San Francisco | 2026-01-03T00:00:00.000+0000 | 16.3 | 12.8 | 13.8 | 0.2 | null | null | null |
| 2 | San Francisco | 2026-01-04T00:00:00.000+0000 | 12.9 | 9.3 | 11.6 | 4.8 | null | null | null |
| 3 | San Francisco | 2026-01-05T00:00:00.000+0000 | 11.9 | 9.4 | 10.8 | 30.9 | null | null | null |
| 4 | San Francisco | 2026-01-06T00:00:00.000+0000 | 14.9 | 9.3 | 11.5 | 1 | null | null | null |
| 5 | San Francisco | 2026-01-07T00:00:00.000+0000 | 14.1 | 8.5 | 10.7 | 0 | null | null | null |
| 6 | San Francisco | 2026-01-08T00:00:00.000+0000 | 12.4 | 6.3 | 9 | 0 | null | null | null |
| 7 | San Francisco | 2026-01-09T00:00:00.000+0000 | 13.1 | 5 | 8.4 | 0 | null | null | null |
| 8 | San Francisco | 2026-01-10T00:00:00.000+0000 | 13.9 | 4.2 | 8.3 | 0 | null | null | null |
| 9 | San Francisco | 2026-01-11T00:00:00.000+0000 | 14.3 | 5 | 8.8 | 0 | null | null | null |
| 10 | San Francisco | 2026-01-12T00:00:00.000+0000 | 14.7 | 5.1 | 8.9 | 0 | null | null | null |
| 11 | San Francisco | 2026-01-13T00:00:00.000+0000 | 15.9 | 5.5 | 9.5 | 0 | null | null | null |
| 12 | San Francisco | 2026-01-14T00:00:00.000+0000 | 16.3 | 6.1 | 10.3 | 0 | null | null | null |
| 13 | San Francisco | 2026-01-15T00:00:00.000+0000 | 17 | 6.8 | 10.9 | 0 | null | null | null |
| 14 | San Francisco | 2026-01-16T00:00:00.000+0000 | 19.1 | 6.2 | 12 | 0 | null | null | null |
| 15 | San Francisco | 2026-01-17T00:00:00.000+0000 | 17.4 | 7.4 | 11.3 | 0 | null | null | null |
| 16 | San Francisco | 2026-01-18T00:00:00.000+0000 | 17.2 | 8.7 | 11.8 | 0 | null | null | null |
| 17 | San Francisco | 2026-01-19T00:00:00.000+0000 | 16.5 | 6.7 | 10.7 | 0 | null | null | null |
| 18 | San Francisco | 2026-01-20T00:00:00.000+0000 | 13.1 | 7.4 | 9.6 | 0 | null | null | null |
| 19 | San Francisco | 2026-01-21T00:00:00.000+0000 | 12.4 | 8 | 10 | 0 | null | null | null |
| 20 | San Francisco | 2026-01-22T00:00:00.000+0000 | 16.7 | 8 | 11.4 | 0 | null | null | null |
| 21 | San Francisco | 2026-01-23T00:00:00.000+0000 | 12.6 | 7.7 | 9.9 | 0 | null | null | null |
| 22 | San Francisco | 2026-01-24T00:00:00.000+0000 | 18.5 | 7.4 | 11.4 | 0 | null | null | null |
| 23 | San Francisco | 2026-01-25T00:00:00.000+0000 | 12.3 | 5.1 | 8.5 | 0 | null | null | null |
| 24 | San Francisco | 2026-01-26T00:00:00.000+0000 | 14.5 | 6.3 | 9.5 | 0 | null | null | null |
| 25 | San Francisco | 2026-01-27T00:00:00.000+0000 | 15.3 | 6 | 10.5 | 0 | null | null | null |
| 26 | San Francisco | 2026-01-28T00:00:00.000+0000 | 17 | 9.8 | 12.7 | 0 | null | null | null |
| 27 | San Francisco | 2026-01-29T00:00:00.000+0000 | 17.5 | 7.6 | 11.5 | 0 | null | null | null |
| 28 | San Francisco | 2026-01-30T00:00:00.000+0000 | 18.2 | 7.7 | 11.6 | 0 | null | null | null |
| 29 | San Francisco | 2026-01-31T00:00:00.000+0000 | 18.3 | 7.1 | 11.8 | 0 | null | null | null |
| 30 | San Francisco | 2026-02-01T00:00:00.000+0000 | 19 | 7.4 | 12.2 | 0 | null | null | null |
| 31 | San Francisco | 2026-02-02T00:00:00.000+0000 | 20.5 | 6.2 | 12.3 | 0 | null | null | null |
| 32 | San Francisco | 2026-02-03T00:00:00.000+0000 | 20.3 | 7.2 | 12.8 | 0 | null | null | null |

Fig. 3 ANALYTICS.LAB1CITY_WEATHER_FINAL Table

*Table 4 : RAW.LAB1ETL*

| Column Name | Data Type | Description | Constraint |
|---|---|---|---|
| LATITUDE | FLOAT | Latitude of the city | Part of Primary Key |
| LONGITUDE | FLOAT | Longitude of the city | Part of Primary Key |
| DATE | DATE | Weather observation date | Part of Primary Key |
| TEMP_MAX | FLOAT | Daily maximum temperature | - |
| TEMP_MIN | FLOAT | Daily minimum temperature | - |
| TEMP_MEAN | FLOAT | Daily mean temperature | - |
| PRECIPITATION | FLOAT | Daily total precipitation | - |
| WEATHER_CODE | VARCHAR(3) | Weather condition | - |

| | | | |
|---|---|---|---|
| | | code from API | |
| CITY | VARCHAR(100) | City name | Part of Primary Key |

USER_DB_COBRA / RAW / LAB1ETL

Load Data

Table TRAINING_ROLE 3 minutes ago 180 15.0KB

Table Details Columns Data Preview Copy History Data Quality PREVIEW Lineage

COBRA_QUERY_WH 100 of 180 Rows • Updated just now

| | LATITUDE | LONGITUDE | DATE | TEMP_MAX | TEMP_MIN | TEMP_MEAN | PRECIPITATION | WEATHER_CODE | CITY |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 37.5485 | -121.9886 | 2026-01-03 | 17 | 12.7 | 14.1 | 0.4 | 51 | Fremont |
| 2 | 37.5485 | -121.9886 | 2026-01-04 | 12.6 | 10.4 | 11.8 | 20.2 | 65 | Fremont |
| 3 | 37.5485 | -121.9886 | 2026-01-05 | 13.8 | 9.2 | 11.4 | 1.5 | 51 | Fremont |
| 4 | 37.5485 | -121.9886 | 2026-01-06 | 16.7 | 10 | 12.8 | 0 | 3 | Fremont |
| 5 | 37.5485 | -121.9886 | 2026-01-07 | 14.3 | 8.9 | 10.9 | 0 | 3 | Fremont |
| 6 | 37.5485 | -121.9886 | 2026-01-08 | 13.8 | 5.8 | 9.4 | 0.2 | 51 | Fremont |
| 7 | 37.5485 | -121.9886 | 2026-01-09 | 15.7 | 4.2 | 8.5 | 0 | 0 | Fremont |
| 8 | 37.5485 | -121.9886 | 2026-01-10 | 15.2 | 2.7 | 8.3 | 0 | 3 | Fremont |
| 9 | 37.5485 | -121.9886 | 2026-01-11 | 15.8 | 2.2 | 8.6 | 0 | 3 | Fremont |
| 10 | 37.5485 | -121.9886 | 2026-01-12 | 17.6 | 3.8 | 9.3 | 0 | 2 | Fremont |
| 11 | 37.5485 | -121.9886 | 2026-01-13 | 18.3 | 3 | 9.7 | 0 | 2 | Fremont |
| 12 | 37.5485 | -121.9886 | 2026-01-14 | 19 | 3.7 | 10.1 | 0 | 3 | Fremont |
| 13 | 37.5485 | -121.9886 | 2026-01-15 | 20.8 | 4.4 | 11.8 | 0 | 2 | Fremont |
| 14 | 37.5485 | -121.9886 | 2026-01-16 | 21.7 | 4.9 | 12.4 | 0 | 1 | Fremont |
| 15 | 37.5485 | -121.9886 | 2026-01-17 | 20.6 | 3.7 | 11.8 | 0 | 3 | Fremont |
| 16 | 37.5485 | -121.9886 | 2026-01-18 | 20.5 | 5.2 | 12 | 0 | 3 | Fremont |
| 17 | 37.5485 | -121.9886 | 2026-01-19 | 19.5 | 3.6 | 11 | 0 | 3 | Fremont |
| 18 | 37.5485 | -121.9886 | 2026-01-20 | 16.7 | 6 | 10.5 | 0 | 3 | Fremont |
| 19 | 37.5485 | -121.9886 | 2026-01-21 | 15 | 7 | 10.8 | 0 | 3 | Fremont |
| 20 | 37.5485 | -121.9886 | 2026-01-22 | 18.7 | 7.5 | 11.9 | 0 | 3 | Fremont |
| 21 | 37.5485 | -121.9886 | 2026-01-23 | 12.4 | 8.3 | 10.3 | 0 | 45 | Fremont |
| 22 | 37.5485 | -121.9886 | 2026-01-24 | 18.7 | 6.5 | 11.7 | 0 | 45 | Fremont |
| 23 | 37.5485 | -121.9886 | 2026-01-25 | 14.4 | 4.7 | 9.4 | 0 | 3 | Fremont |
| 24 | 37.5485 | -121.9886 | 2026-01-26 | 18.4 | 7.3 | 11.4 | 0 | 3 | Fremont |
| 25 | 37.5485 | -121.9886 | 2026-01-27 | 18.4 | 5.2 | 11.3 | 0 | 3 | Fremont |
| 26 | 37.5485 | -121.9886 | 2026-01-28 | 18.9 | 9.5 | 13.5 | 0 | 3 | Fremont |
| 27 | 37.5485 | -121.9886 | 2026-01-29 | 18.3 | 5.1 | 11.8 | 0 | 1 | Fremont |
| 28 | 37.5485 | -121.9886 | 2026-01-30 | 19.8 | 6.1 | 12.3 | 0 | 3 | Fremont |
| 29 | 37.5485 | -121.9886 | 2026-01-31 | 20.3 | 6.1 | 12.9 | 0 | 3 | Fremont |
| 30 | 37.5485 | -121.9886 | 2026-02-01 | 19.2 | 7.1 | 12.4 | 0 | 2 | Fremont |
| 31 | 37.5485 | -121.9886 | 2026-02-02 | 20.5 | 5.8 | 12.5 | 0 | 0 | Fremont |
| 32 | 37.5485 | -121.9886 | 2026-02-03 | 21.3 | 4.2 | 12.8 | 0 | 1 | Fremont |

Fig. 4 RAW.LAB1ETL Table

## *B. Airflow DAGs (Python Code Links)*

1) *ETL:*https://github.com/sangahlee0/Data226_labs/blob/ce495c23de6d14fc32f427bfaebce11ee9b2b8d2/Lab1_ETL.py
2) *Forecast:*https://github.com/sangahlee0/Data226_labs/blob/ce495c23de6d14fc32f427bfaebce11ee9b2b8d2/Lab1_forecast.yesspy
3) *SQL Queries :* https://github.com/sangahlee0/Data226_labs/blob/main/lab_2.sql

## C. Airflow Web UI Screenshots

### 1) Screenshot: DAGs Overview (Two Pipelines):

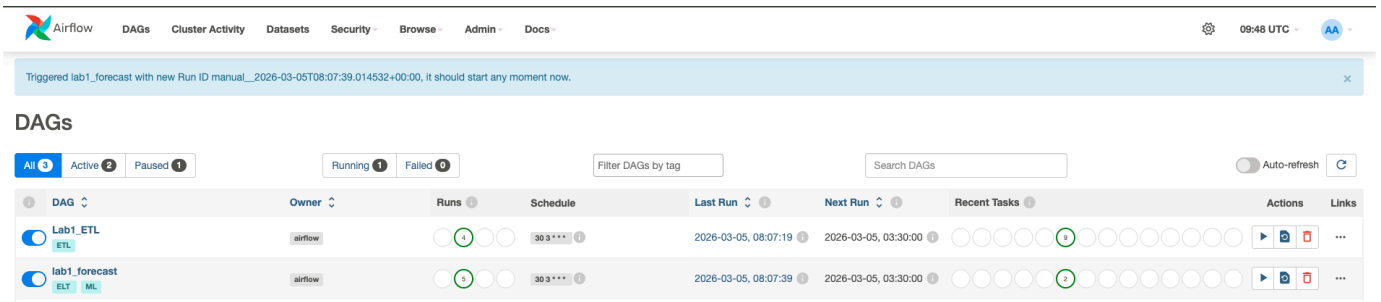


Fig. 5 DAGs Overview Page

### 2) Screenshot: DAG Run Ordering / Trigger After ETL:

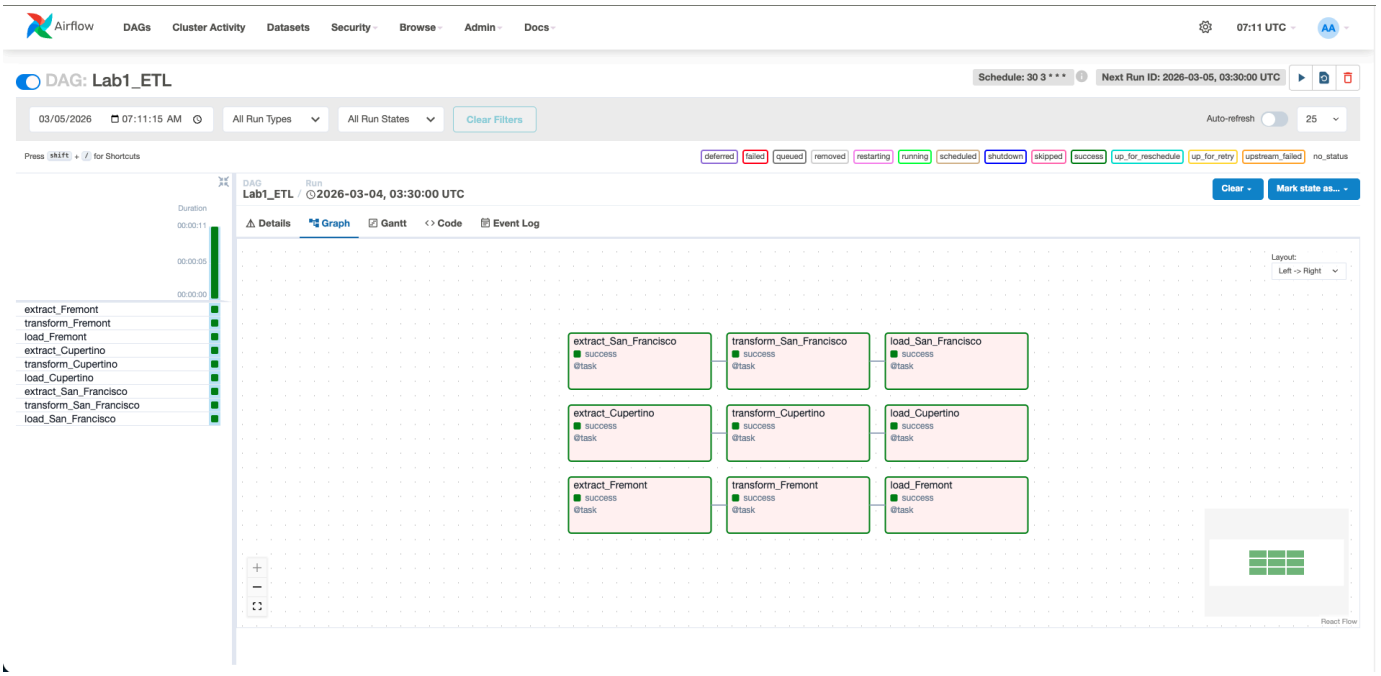


Fig. 6 DAG ETL after ETL

*D. Airflow Connections and Variables (Evidence)*

*1) Connections:*

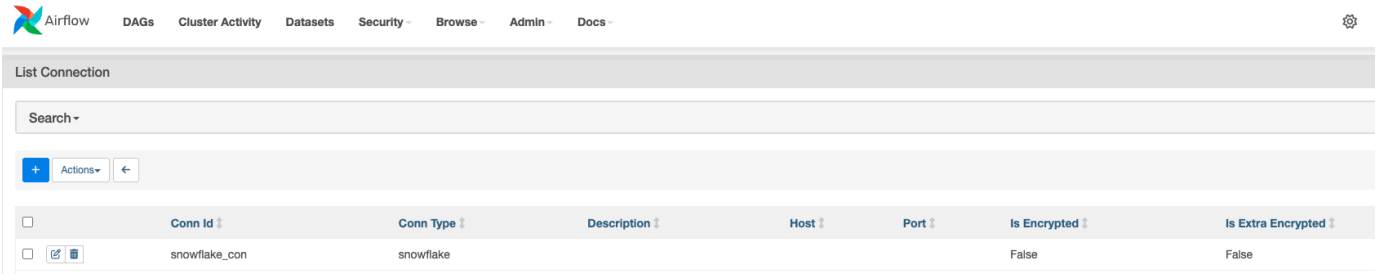


Fig. 7 Airflow Connection

*2) Variables:*

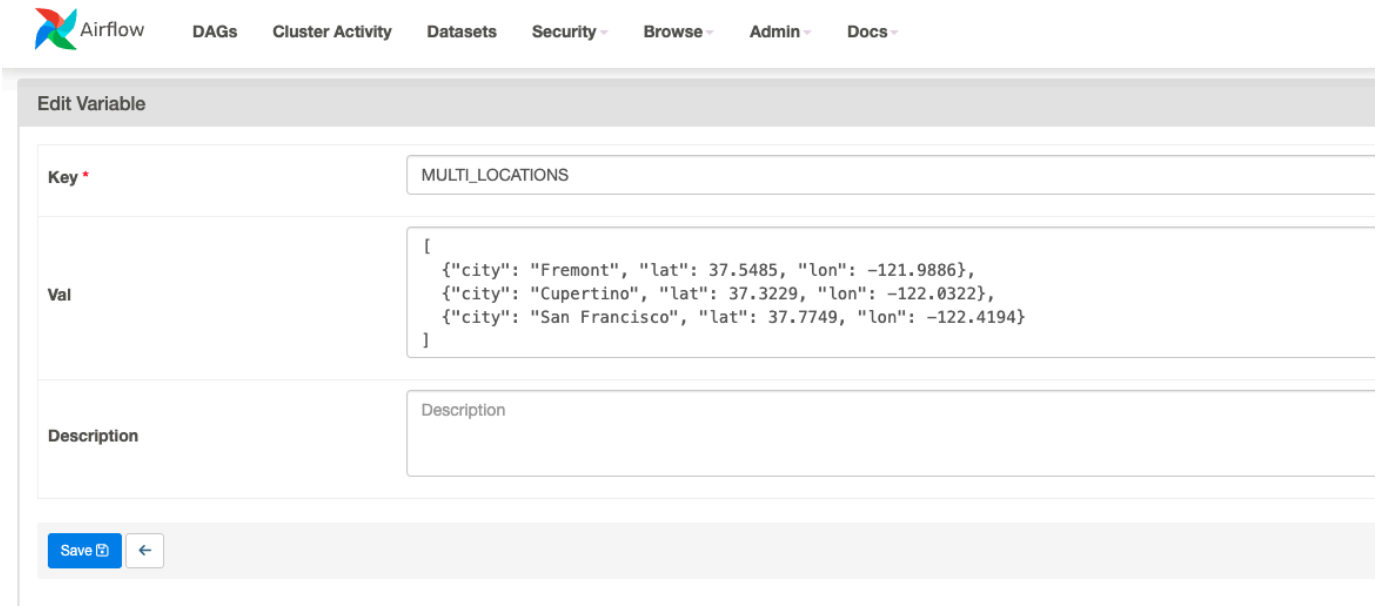


Fig. 8 Airflow Variables

## *E. Final Table Creation (UNION of ETL + Forecast Tables)*

### *1) Final Table*

```
# Build final table
create_final_table_sql = f"""
CREATE OR REPLACE TABLE {final_table} AS
    SELECT CITY, DATE,
           TEMP_MAX as actual_max,
           TEMP_MIN,
           TEMP_MEAN AS actual_mean,
           PRECIPITATION,
           NULL AS forecast, NULL AS lower_bound, NULL AS upper_bound
    FROM {train_input_table}
UNION ALL
    SELECT REPLACE(series, '\"', '') as CITY,
           ts as DATE,
           NULL AS actual_max, NULL AS TEMP_MIN, NULL AS actual_mean, NULL AS PRECIPITATION,
           forecast, lower_bound, upper_bound
    FROM {forecast_table};
"""
cur.execute(create_final_table_sql)

print(f"Forecast table: {forecast_table}")
print(f"Final table: {final_table}")
```

Fig. 9 SQL Code Unionizing the tables

USER_DB_COBRA / ADHOC / LAB1_CITY_WEATHER_FORECAST

Load Data

Table | TRAINING_ROLE | 2 minutes ago | 21 | 3.0KB

Table Details | Columns | Data Preview | Copy History | Data Quality PREVIEW | Lineage

COBRA_QUERY_WH | 21 Rows • Updated just now

| | SERIES | TS | FORECAST | LOWER_BOUND | UPPER_BOUND |
|---|---|---|---|---|---|
| 1 | "Fremont" | 2026-03-04T00:00:00.000+0000 | 18.051441879 | 11.510194664 | 24.138091479 |
| 2 | "Fremont" | 2026-03-05T00:00:00.000+0000 | 19.717136386 | 13.910337387 | 25.624483329 |
| 3 | "Fremont" | 2026-03-06T00:00:00.000+0000 | 20.636909883 | 14.651368974 | 26.466676 |
| 4 | "Fremont" | 2026-03-07T00:00:00.000+0000 | 20.523309059 | 15.056976316 | 26.554834242 |
| 5 | "Fremont" | 2026-03-08T00:00:00.000+0000 | 19.77876606 | 13.9825953 | 26.488106036 |
| 6 | "Fremont" | 2026-03-09T00:00:00.000+0000 | 18.803848426 | 12.341163653 | 24.592748587 |
| 7 | "Fremont" | 2026-03-10T00:00:00.000+0000 | 18.946396041 | 13.345488698 | 25.174196517 |
| 8 | "Cupertino" | 2026-03-04T00:00:00.000+0000 | 19.9353741 | 14.490069674 | 25.024857518 |
| 9 | "Cupertino" | 2026-03-05T00:00:00.000+0000 | 21.760334228 | 16.823134365 | 26.77625775 |
| 10 | "Cupertino" | 2026-03-06T00:00:00.000+0000 | 23.183141477 | 18.050811982 | 28.193535646 |
| 11 | "Cupertino" | 2026-03-07T00:00:00.000+0000 | 23.141347531 | 18.367224136 | 28.357856401 |
| 12 | "Cupertino" | 2026-03-08T00:00:00.000+0000 | 22.458005568 | 17.382358656 | 28.248392949 |
| 13 | "Cupertino" | 2026-03-09T00:00:00.000+0000 | 19.852304706 | 14.2152161 | 24.962008749 |
| 14 | "Cupertino" | 2026-03-10T00:00:00.000+0000 | 20.476028204 | 15.476543127 | 25.96618658 |
| 15 | "San Francisco" | 2026-03-04T00:00:00.000+0000 | 16.78025105 | 10.486706604 | 22.636412493 |
| 16 | "San Francisco" | 2026-03-05T00:00:00.000+0000 | 18.118714598 | 12.531806425 | 23.802362911 |
| 17 | "San Francisco" | 2026-03-06T00:00:00.000+0000 | 18.060921178 | 12.302039813 | 23.669926694 |
| 18 | "San Francisco" | 2026-03-07T00:00:00.000+0000 | 18.752633061 | 13.49329844 | 24.555757527 |
| 19 | "San Francisco" | 2026-03-08T00:00:00.000+0000 | 17.633561413 | 12.056879007 | 24.088833469 |
| 20 | "San Francisco" | 2026-03-09T00:00:00.000+0000 | 17.107976079 | 10.890019116 | 22.677663201 |
| 21 | "San Francisco" | 2026-03-10T00:00:00.000+0000 | 16.892307294 | 11.503494126 | 22.884274605 |

Fig. 10 ADHOC.LAB1_CITY_WEATHER_FORECAST Table

### *F. SQL Transactions + try/except (Evidence)*

```
try:
    cur.execute("BEGIN;")

    cur.execute(f"""CREATE TABLE IF NOT EXISTS {target_table} (
        latitude FLOAT,
        longitude FLOAT,
        date DATE,
        temp_max FLOAT,
        temp_min FLOAT,
        temp_mean FLOAT,
        precipitation FLOAT,
        weather_code VARCHAR(3),
        city VARCHAR(100),
        PRIMARY KEY (latitude, longitude, date, city)
        );
    """)
    cur.execute(f"""DELETE FROM {target_table}""")

    insert_sql = f"""INSERT INTO {target_table} (latitude, longitude, date, temp_max, temp_min, temp_mean, precipitation, weather_code, cit

    data = [
        (   r['latitude'],
            r['longitude'],
            r['date'],
            r['temp_max'],
            r['temp_min'],
            r['temp_mean'],
            r['precipitation'],
            str(r['weather_code']),
            r['city']
        )
        for r in records
    ]
    cur.executemany(insert_sql, data)

    cur.execute("COMMIT;")
    print(f"Loaded {len(records)} records into {target_table}")

except Exception as e:
    cur.execute("ROLLBACK;")
    print(e)
    raise e
```

Fig. 11 SQL Code showing try/except blocks

## V. Lessons Learned and Conclusion

There is a big importance of how automation plays a key role in running tasks and forecasting, which has been shown through this project. A primary advantage of using Docker, especially while working in a group, is that the platform made it easier to run in the Airflow environment and ensured reproducibility. Airflow UI has allowed easier access to visualize the scheduled tasks, and to debug errors as results show exactly where a failure occurred in the pipeline through real-time observability. As for data collection, DAG coordinated the weather data collection by determining the schedule, allowing the SQL queries to do their work in forecasting, and extracting, transforming, and loading the data per location. These systems played a key role in analyzing the data and forecasting. It is especially easy to add new locations through Airflow, and take into considerations of other locations.

Through this experience, the significance of DAG, data pipeline, and databases are illustrated as connections integral in creating automated data analyses.